

Spiel Projekt

Niklas Hainz

05.06.2026

Praktische Informatik

Inhaltsverzeichnis

1	Einleitung	1
2	Planung	1
2.1	Produktskizze	1
2.1.1	Ziel des Spieles	1
2.1.2	Genre des Spiels	1
2.1.3	Grobe Struktur der Benutzeroberfläche	1
2.1.4	Aufgaben und Vorgehen des Spielers	1
2.1.5	Spielende	1
2.2	Prototypen	1
2.2.1	Spielbeginn	2
2.2.2	Typische Szene im Spiel	2
2.2.3	Spielende	2
2.3	Versionierung	2
3	Entwurf	3
3.1	Struktogramm einer Methoden	3
4	Durchführung	3
4.1	Beschreibung Ablauf	3
4.1.1	Ideenfindung	3
4.1.2	Erstes Level	3
4.1.3	Blasen	3
4.1.4	Gegner	3
4.1.5	Spiellogik	3
4.2	Beschreibung des Spielablaufs	4
4.3	Fehlerprotokoll und Maßnahmen zur Behebung	4
4.4	Kommentierter Quelltext	5
4.4.1	Gegner	5
4.4.1.1	Gegner Bewegung	5
4.4.1.2	Flip vom Gegner	5
4.4.1.3	Über Loch Springen	6
4.4.2	Spieler	7
4.4.2.1	Bewegung	7
4.4.2.2	Animation	7
4.4.2.3	Deaktivierung von Hitbox beim Springen	8
4.4.3	Hauptzehne	8
4.4.3.1	Gegner Erscheinen Lassen	8
4.4.3.2	Blase In Hauptzehne einfügen	8
4.4.4	Game state	9
4.4.4.1	Ändern Level	9
4.5	Begründung von Abweichungen gegenüber Planung	9
4.6	Screenshots	9
5	Fazit	10
6	Quellen	10

List of Figures

1	Struktogramm Change Level	3
2	Gegner Neben Spieler	9
3	Spieler der Auf gegner Schießt	9

1 Einleitung

Bubble Bobble ist ein sehr altes Original (Bubble Bobble aus dem Jahr 1986 entwickelt von Taito) inspiriertes Spiel, das diese nachahmt. Es geht darum, Gegner in einem stationären Level zu vernichten, was man mit Blasen als Spieler machen kann.

2 Planung

2.1 Produktskizze

Das geplante Spiel soll den gleichen Aufbau und Ablauf wie das Spiel "Bubble Bobble" haben: Das Spiel besteht aus mehreren Levels, welche alle eine verschiedene Anzahl und Typen an Gegnern beinhalten. Diese Gegner müssen alle besiegt werden, um in das nächste Level zu kommen. Die Gegner sind besiegbare Blasen, die vom Spieler aus geschossen werden können.

2.1.1 Ziel des Spieles

Das Ziel des Spieles ist es, alle Level zu bestehen.

2.1.2 Genre des Spiels

Das Spiel ist das Genre Jump 'n' Run.

2.1.3 Grobe Struktur der Benutzeroberfläche

Oben in der Mitte soll immer der Highscore und der Score vom ersten Spieler links und der Score vom zweiten Spieler unten links sollte noch die Leben in Form von Blasen angezeigt werden.

2.1.4 Aufgaben und Vorgehen des Spielers

Der Spieler muss alle Gegner besiegen, bis er ins nächste Level kommt.

2.1.5 Spielende

Das Spielende tritt ein, wenn Spieler eins und Spieler zwei keine Leben mehr haben, dann wird ein "Game over"-Screen eingeblendet.

2.2 Prototypen



Die roten Rechtecke in der Mitte sind Gegner. Das blaue und das grüne Rechtecke sind Spieler eins (Grün) und Spieler zwei (Blau). Die Rechtecke links und rechts sind für ein Bonus-Leben. Oben in der Mitte ist der Highscore, der der höchste Spielerpunktzahl oder wenn der einer der aktuellen Spieler einen höheren Highscore hat, dann wird dieser ersetzt und geupdated.

2.2.1 Spielbeginn

Zu Spiel beginn muss entweder eine virtuelle Münze eingeworfen werden(Start Taste) oder das Spiel Starte direkt

2.2.2 Typische Szene im Spiel

Spieler kämpft gegen Gegner um ins Nächste Level zu kommen. Ggf. Zwei Spieler.

2.2.3 Spielende

Spiel ende Trifft eine wenn beide Spieler keine Leben mehr haben. Dann würde der “Game Over” Screen eintreten

2.3 Versionierung

Versionierung mit Git. Planung sind mindesten 20 Level wünschenswert wahren mehr bis alle 100 und einföhrung von den Bonbons als Powerups auch hier wäres wünschens wert wenn es noch mehr Items hinzugefügt werden. Selbst erstelle Sprites.

3 Entwurf

3.1 Struktogramm einer Methoden

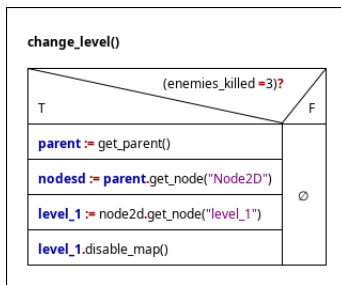


Figure 1: Struktogramm Change Level

4 Durchführung

4.1 Beschreibung Ablauf

4.1.1 Ideenfindung

Die Durchführung begann mit der Ideenfindung, wo recht schnell die Idee kam ein Arcade Game aus den 80er 90er nach zu Programmieren. Als alternative Wahl stand noch das Spiel Pang. Es wurde sich aber dagegen entschieden.

4.1.2 Erstes Level

Danach wurde Angefangen der Erste teil der Dokumentation zu Schreiben. Also die Überpunkte und Unterpunkte 1 und 2 Geschrieben. Danach wurde Angefangen sich mit Godot auseinander zu setzen. Nachdem sich mit Godot genug auseinander gesetzt wurde, wurde angefangen das erste Level bzw. die Texturen für das erste Level zu erstellen. Zum erstellen des ersten Leveles wurde in Godot die Tilemap Layer genutzt. Außerdem wurde die Kamera hier erstellt.

4.1.3 Blasen

Mit der Basis des ersten Levels als Testumgebung wurden danach die Funktionalität des Schießens der Blasen mit dem Spieler hinzugefügt. Die Blase wurde erst als RigidBody implementiert, da dadurch der Spieler und die Umgebung damit interagieren könnten. Dies führte aber zu unerwarteten Verhalten wodurch dies zur Area 2D umgeändert wurde.

4.1.4 Gegner

Nachdem die Blase programmiert war wurde angefangen einen Gegner zu Programmieren, als erstes seine Bewegung sodass er nach Rechts läuft und wenn er gegen eine Wand prallt, umkehrt und in die andere Richtung läuft. Danach wurde das Springen auf eine höhere Plattform programmiert, wenn der Spieler über dem Gegner ist. Danach wurde noch implementiert dass der Gegner über Löcher im Boden Springen kann, bzw. entscheidet ob er darüber springt oder runter fällt um zum Spieler zu kommen.

4.1.5 Spiellogik

Danach wurde angefangen ein Script zu schreiben das, dass Spiel Managet also es setzt den Score oder Startet neue Level an. Danach wurden sich darauf Fokussiert weitere Level erstellt

4.2 Beschreibung des Spielablaufs

Der Spielablauf besteht darin, dass der Spieler im Level alle Gegner tötet um ins nächste Level zu kommen das Spiel endet wenn der Spieler stirbt also alle Leben verliert oder wenn er alle Level besiegt hat.

4.3 Fehlerprotokoll und Maßnahmen zur Behebung

Einer der auffälligsten Fehler im Spiel war die fehlerhafte Animation, wenn der Spieler gleichzeitig nach Links und nach Rechts Laufen gedrückt hat. Lösen ließ sich das Problem mit einem ändern der Bedingung in den Playeranimationen. Der Code Fragte zuerst nur nach Bodenkontakt und Richtung in welche gedrückt wird ab. Dies führte jedoch dazu, dass in diesem Fall dauerhaft eine Laufanimation gespielt wurde aber der Charakter nicht bewegt wurde. lösen konnte man das Problem indem die Bedingung erweiter wurde um eine abfrage, ob die andere Laufrichtung auch gedrückt ist.

4.4 Kommentierter Quelltext

4.4.1 Gegner

4.4.1.1 Gegner Bewegung

Die Folgende Methode ist für die Bewegung des Gegners da. Nach der Funktion wird gescheckt, ob ob der Spieler nicht auf dem Boden ist wenn dies so ist wird Gravitaton auf ihn angewant. Danach wird der Parent ermittelt und darauf hin wird der Spieler mit dem Parent ermittelt. Wenn am Boden dann bewege dich mit deiner Laufgeschwindigkeit * die Richtung die Richtung, wobei die Variable Richtung die Richtung bestimmt. `move_and_slide()` Methode von Godot macht aus `velocity` Bewegung.

```
func _physics_process(delta: float) -> void:
    # Add the gravity.
    #wenn Spieler auf dem Boden dann wende Gravitaton an
    if not is_on_floor():
        velocity += get_gravity() * delta / 8

    #Bestimme Parent und darueber Spieler
    var parent = get_parent()
    var player = parent.get_node("Player")

    # Bestimmt ob Gelaufen werden soll , oder ob Gestoppt wird
    if is_on_floor():
        velocity.x = SPEED * richtung
    elif velocity.y < 0 and player.position.y > self.position.y -
JUMP_VELOCITY:
        velocity.x = 0

    #Noeting fuer Bewegung
    move_and_slide()
```

4.4.1.2 Flip vom Gegner

Wenn der Gegner gegen die Wand läuft wird er einmal Gedreht. Dies passiert über den Raycast

```
func flip() -> void:
    #Wenn Raycast Kollidiert dann fuehre aus
    if wall_collsion.is_colliding() == true:
        richtung *= -1 #aendert vorzeichen von der Richtung
        sprite.flip_h = !sprite.flip_h # dreht sprite
        wall_collsion.target_position.x *= -1 # dreht Raycast
        check_floor.position.x *= -1 # dreht Raycast
```


4.4.1.3 Über Loch Springen

```
func jump_over_hole() -> void:
    var parent = get_parent()
    var player = parent.get_node("Player")
    #Aendern zu Int da Float zu genau.
    var player_position:int = player.position.y
    var zen_chan_position:int = self.position.y

    if !check_floor.is_colliding() and is_on_floor() and
        player_position < zen_chan_position - 20 or
        !check_floor.is_colliding() and is_on_floor() and
        player_position == zen_chan_position:
        velocity.y = JUMP_VELOCITY / 2
```

4.4.2 Spieler

4.4.2.1 Bewegung

Das Script ist Zur Bewegung da und guckt ob der Jeweilige Knopf gedrückt wurde um dann den Spieler in die Richtung zu bewegen

```
func _physics_process(delta: float) -> void:
    # Fuegt Gravitation zu.
    if not is_on_floor():
        velocity += get_gravity() * delta / 8

    # Fuers Springen
    if Input.is_action_just_pressed("move_up") and is_on_floor():
        velocity.y = JUMP_VELOCITY

    # Bildet die Summe von Negativ und Positiv Bewegung und bewegt dann.
    var direction := Input.get_axis("move_left", "move_right")
    if direction:
        velocity.x = direction * SPEED
    else:
        velocity.x = move_toward(velocity.x, 0, SPEED)
    #Wichtig fuer die Bewegung
    move_and_slide()
```

4.4.2.2 Animation

Diese Script gibt dem Spieler Animationen dies ist mit If-abfragen gelöst die abfragen ob der Gegebene Knopf für die Gegebene Richtung gedrückt wurde.

```
func animations() -> void:
    # Wenn Rechts gedrueckt dann Drehe Sprite
    if Input.is_action_pressed("move_right") and
    not Input.is_action_pressed("move_left"):
        _animated_sprite.flip_h = true
        emit_signal("right")
        _animated_sprite.play("walk")
    # Ansonsten wenn Knopf Links gedrueckt dann Flip nicht bzw. zurueck
    elif Input.is_action_pressed("move_left") and
    not Input.is_action_pressed("move_right"):
        _animated_sprite.flip_h = false
        emit_signal("left")
        _animated_sprite.play("walk")
    else:
        _animated_sprite.stop()
```

4.4.2.3 Deaktivierung von Hitbox beim Springen

Deaktiviert die Hitbox when Spieler sich nach oben Bewegt

```
func handle_collision_deactivating_when_jumping():  
    # Wenn Geschwindigkeit in y richtung kleiner 0 (springen) dann  
    Deaktiviere Hitbox  
    if velocity.y < 0 :  
        set_collision_mask_value(4, false)  
    else :  
        set_collision_mask_value(4, true)
```

4.4.3 Hauptzehne

4.4.3.1 Gegner Erscheinen Lassen

Lädt die Scene Rein setzt die Position fest und Generiert sie.

```
func spawn_enemie():  
    # Laedt gegner scene in Variable  
    var enemy_scene = preload("res://scenes/zen_chan.tscn")  
    # instantiert Gegner scene in variable  
    var enemy = enemy_scene.instantiate()  
    #Setzt position  
    enemy.position = enemy_spawn_point.position  
    #Fuegt gegner als Kind zur Hauptzehne hinzu  
    add_child(enemy)
```

4.4.3.2 Blase In Hauptzehne einfügen

Fügt die Blase in die Hauptszene hinzu wenn dies Vom Spieler ausgeht

```
func _on_character_body_2d_create_bubble() -> void:  
    #erstellt die Blase  
    var bubble = scene.instantiate()  
    #Setzt Blase position auf Spieler Position  
    bubble.position = player.position  
    #Entscheidet in Welche Richtung die Blase Generiert wird  
    if direction == "right":  
        bubble.position.x = player.position.x +15  
    elif direction == "left":  
        bubble.position.x = player.position.x -15  
    add_child(bubble)
```

4.4.4 Game state

4.4.4.1 Ändern Level

```
if enemies_killed == 3:
    var parent = get_parent()
    var node2d = parent.get_node("Node2D")
    var level_1 = node2d.get_node("level_1")
    level_1.disable_map()
```

4.5 Begründung von Abweichungen gegenüber Planung

Alle Abweichungen vom Plan waren entweder Sehr schwer oder zu Aufwendig zu implementiert oder aus Zeitgründen nicht gemacht worden.

4.6 Screenshots



Figure 2: Gegner Neben Spieler

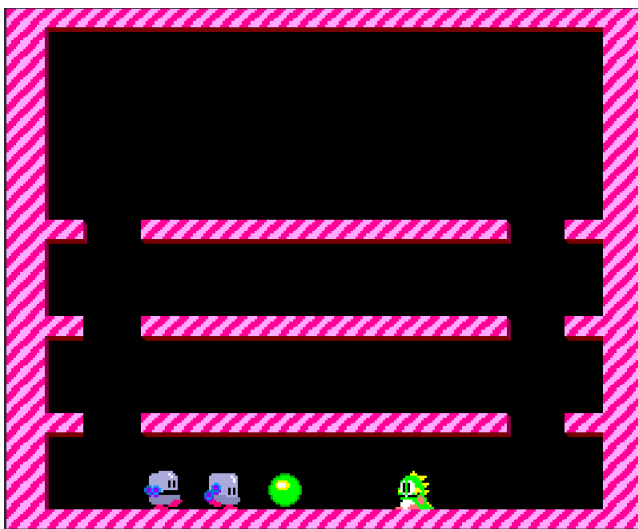


Figure 3: Spieler der Auf gegner Schießt

5 Fazit

Das Zeitmanagent war schlecht und Ziele zu groß.

6 Quellen